W2 — JavaScript ve ES6 | Sınav Çalışma Dosyası

Kaynak: zeki-ozen/MobileAppDevelopment / W2
Kapsam: Depodaki `W2/README.md` ve `W2/topics/00–10` numaralı dersler.
Kullanım: Açıklama + örnek kod + beklenen sonuç + sınavda dikkat edilecek noktalar. Kodlar tarayıcı konsolunda veya Node.js ortamında denenebilir.

> **Önemli ayrım:** Bu hafta **JavaScript dili** anlatılır. Buradaki `console.log()` örnekleri Expo Snack'teki `App.js` ekranına yazı basmaz; mobil ekranda yazı göstermek için daha sonraki haftalarda `<Text>` kullanılır.

1. JavaScript nedir, nerede çalışır?

JavaScript; değişken, koşul, döngü, fonksiyon ve nesne yazmayı sağlayan programlama dilidir. Tarayıcıda HTML ile, Node.js ile sunucu/terminal ortamında, React Native'de uygulama mantığında kullanılır. **ES6**; `let`, `const`, ok fonksiyonları, modüller, destructuring, spread gibi modern sözdizimlerini yaygınlaştıran 2015 standart sürümüdür.

```
console.log('Merhaba JavaScript');
const ad = 'Ünal';
console.log(`Merhaba ${ad}`); // Merhaba Ünal
```

- **Tarayıcı:** `F12` → Console'a yazıp çalıştır.
- **Node.js:** `node deneme.js` komutuyla `.js` dosyasını çalıştır.
- **HTML içinde:** `<script src="app.js" defer></script>`; `defer` HTML ayrıştırması tamamlandıktan sonra script'in çalışmasını sağlar (klasik dış script).
- **Modül içinde:** `<script type="module" src="main.js"></script>`; modüller kendi kapsamına sahiptir, varsayılan olarak gecikmeli yürütülür.

```
<!doctype html>
<html lang="tr">
  <head><meta charset="UTF-8"><script src="app.js" defer></script></head>
  <body><h1>JavaScript denemesi</h1></body>
</html>
```

2. Değişkenler: `let`, `const`, `var`

İfade	Yeniden değer atanır mı?	Blok kapsamı	Tercih
`const`	Hayır	Evet	Değişmeyecek bağlar için ilk tercih
`let`	Evet	Evet	Sayaç/güncellenecek değer

İfade	Yeniden değer atanır mı?	Blok kapsamı	Tercih
`var`	Evet	Hayır (fonksiyon kapsamı)	Eski kodlarda görülebilir

```
const okul = 'Üniversite';
let puan = 40;
puan = 55;            // olur
// okul = 'Lise';    // TypeError: const'a yeniden atama yok

const ogrenci = { ad: 'Ünal' };
ogrenci.ad = 'Ali';   // olur: nesnenin İÇERİĞİ değişti
// ogrenci = {};    // olmaz: const bağını değiştirmeye çalışır
```

Hoisting / Temporal Dead Zone (TDZ): `var` tanımı yukarı taşınmış gibi görülür ve ilk değeri `undefined` olabilir; `let` ve `const` tanımlanmadan önce okunamaz. `const` tanımlanırken ilk değer zorunludur.

```
console.log(a); // undefined
var a = 5;
// console.log(b); // ReferenceError (TDZ)
let b = 8;
```

Sınav tuzağı: `const` ile tanımlanmış bir **diziye** `push()` yapılabilir; yasak olan dizinin değişkene tümden başka dizi olarak atanmasıdır.

3. Veri tipleri ve tip dönüşümü

Tip	Örnek	Not
`string`	`'Merhaba'`	Metin, template literal: `` `Merhaba ${ad}` ``
`number`	`12`, `3.5`, `NaN`	Tamsayı ve ondalık aynı tip
`boolean`	`true`, `false`	Koşullar
`undefined`	`let x;`	Değer henüz verilmemiş
`null`	`let x = null;`	Bilinçli boş değer
`bigint`	`123n`	Büyük tamsayı
`symbol`	`Symbol('id')`	Benzersiz simge
`object`	`{x:1}`, `[1,2]`	Başvuru türü; diziler teknik olarak object

```
console.log(typeof 12);             // 'number'
console.log(typeof '12');           // 'string'
console.log(typeof null);           // 'object' (dilsel tarihsel özellik)
console.log(Array.isArray([1,2]));  // true
console.log(Number('42'));          // 42
console.log(String(42));            // '42'
console.log(Boolean(''));           // false
```

`Number('abc')` sonucu `NaN`; `Number.isNaN(...)` ile kontrol edilir. `null` ile `undefined` farklıdır. Nesneler/diziler karşılaştırılırken **referans** dikkate alınır.

```
console.log([1] === [1]); // false, iki farklı dizi nesnesi
const x = [1]; const y = x;
console.log(x === y);     // true
```

4. Operatörler ve önemli farklar

Operatör	Amaç	Örnek ve sonuç
`+ - * / % **`	Aritmetik	`7 % 3` → `1`, `2 ** 3` → `8`
`=== / !==`	Tip dahil eşitlik	`5 === '5'` → `false`
`==`	Dönüştürerek eşitlik	`5 == '5'` → `true`; genelde kaçın
`&&`	VE, kısa devre	`true && false` → `false`
`||`	VEYA / ilk truthy	`'' || 'varsayılan'` → `'varsayılan'`
`??`	nullish varsayılan	`0 ?? 10` → `0`
`?.`	Güvenli özellik okuma	`kisi?.adres?.il`
`? :`	Üçlü koşul	`not >= 50 ? 'Geçti' : 'Kaldı'`

```
const giris = 0;
console.log(giris || 10); // 10; 0 falsy
console.log(giris ?? 10); // 0; null/undefined değil
console.log(3 + '4');     // '34' (string birleştirme)
console.log(3 - '1');     // 2 (sayısal dönüşüm)
```

Falsy değerler: `false`, `0`, `-0`, `0n`, `''`, `null`, `undefined`, `NaN`. `[]` ve `{}` **truthy** değerlerdir.

5. Koşullar, `switch`, guard clause

```
function notDurumu(not) {
  if (not < 0 || not > 100) return 'Geçersiz not'; // guard
  if (not >= 85) return 'AA';
  else if (not >= 50) return 'Geçti';
  return 'Kaldı';
}
console.log(notDurumu(72)); // Geçti
```

```
const gun = 2;
switch (gun) {
  case 1: console.log('Pazartesi'); break;
  case 2: console.log('Salı'); break;
  default: console.log('Bilinmeyen gün');
}
```

`break` olmazsa normal `switch` içinde sonraki `case` bloklarına da geçilir. `switch` eşleştirmesi katı eşitlik gibi davranır.

6. Döngüler: `for`, `while`, `for...of`, `forEach`

```
for (let i = 1; i <= 3; i++) console.log(i); // 1,2,3
let k = 0;
while (k < 2) { console.log(k); k++; }        // 0,1
const renkler = ['kırmızı','mavi'];
for (const renk of renkler) console.log(renk);
renkler.forEach((renk, index) => console.log(index, renk));
```

- `for...of`: Dizinin **değerlerini** gezer.
- `for...in`: Nesnenin **anahtarlarını** gezer; diziler için genelde `for...of` tercih edilir.
- `break`: Döngüyü bitirir. `continue`: O turu atlar.
- `forEach` içindeki `return`, dıştaki fonksiyondan veya döngüden çıkış gibi davranmaz. Erken durmak gerekiyorsa `for...of` veya `some` / `find` kullan.

7. Fonksiyonlar, arrow function, parametreler

```
function topla(a, b) { return a + b; }
const carp = (a, b) => a * b;
const selamla = (ad = 'Misafir') => `Selam ${ad}`;
console.log(topla(3,4));  // 7
console.log(carp(3,4));   // 12
console.log(selamla());   // Selam Misafir
```

```
const toplam = (...sayilar) => sayilar.reduce((t, s) => t + s, 0);
console.log(toplam(2, 3, 5)); // 10
const kare = x => x * x;       // tek parametrede parantez opsiyonel
const nesneUret = ad => ({ ad }); // nesne döndürürken parantez kullan
```

Dikkat: `() => { x: 1 }` nesne döndürmez; blok olarak değerlendirilir. Ok fonksiyonlarının kendi `this` bağlamı yoktur; dıştaki bağlamı kullanır.

8. Diziler: `map`, `filter`, `find`, `reduce`

```
const sayilar = [1, 2, 3, 4];
const ikiKati = sayilar.map(x => x * 2);         // [2,4,6,8]
const ciftler = sayilar.filter(x => x % 2 === 0); // [2,4]
const ilkBuyuk = sayilar.find(x => x > 2);        // 3
const toplam = sayilar.reduce((t, x) => t + x, 0); // 10
console.log(sayilar.includes(3));                // true
```

Metot	Ne üretir?	Orijinal diziyi değiştirir mi?
`map`	Her elemana karşılık yeni dizi	Hayır
`filter`	Koşulu sağlayanlardan yeni dizi	Hayır
`find`	İlk uygun öğe veya `undefined`	Hayır
`reduce`	Birikimli tek değer (veya yapı)	Hayır
`push`, `pop`, `splice`	Dizinin içeriğini günceller	**Evet**
`slice`	Kesit içeren yeni dizi	Hayır

```
const kisiler = [
  { id: 1, ad: 'Ünal', puan: 75 },
  { id: 2, ad: 'Ayşe', puan: 90 },
  { id: 3, ad: 'Can', puan: 35 },
];
const gecenAdlar = kisiler
  .filter(k => k.puan >= 50)
  .map(k => k.ad);
console.log(gecenAdlar); // ['Ünal','Ayşe']
```

9. Nesneler, destructuring, spread / rest

```
const kullanici = { id: 5, ad: 'Ünal', sehir: 'İstanbul' };
const { ad, sehir } = kullanici;
const liste = [10, 20, 30];
const [birinci, ...kalan] = liste;
const guncel = { ...kullanici, sehir: 'Ankara' };
const eklenmis = [...liste, 40];
```

`...` iki bağlamda kullanılır: **spread** (dizi/nesneyi yaymak) ve **rest** (kalanları toplamak). Nesne spread işlemi **sığ kopyadır**; içteki nesneler aynı referansı paylaşabilir.

10. Modüller: `export` ve `import`

mathUtils.js dosyası:

```
export const PI = 3.14;
export function topla(a, b) { return a + b; }
export default function kare(x) { return x * x; }
```

main.js dosyası:

```
import kare, { PI, topla } from './mathUtils.js';
console.log(topla(2,3), kare(4), PI); // 5 16 3.14
```

- **Named export:** İçe alırken `{ }` kullanılır, ad eşleşir (`as` ile takma ad verilebilir).
- **Default export:** İçe alırken `{ }` kullanılmaz; dosya başına en fazla bir default export vardır.
- Tarayıcıda modül kullanımında `type="module"` gerekir; yerel dosya (`file://`) yerine yerel sunucu daha güvenilirdir.

11. Mini alıştırma — Not listesi (çalışır JS)

Aşağıdaki kod doğrudan Node.js/tarayıcı konsolunda çalışır. `filter` + `map` + `reduce` aynı anda kullanılır:

```
const notlar = [40, 75, 90, 45, 60];
const gecenler = notlar.filter(n => n >= 50);
const etiketler = gecenler.map(n => `Not: ${n}`);
const ortalama = notlar.reduce((t, n) => t + n, 0) / notlar.length;
console.log(gecenler);   // [75,90,60]
console.log(etiketler);  // ['Not: 75','Not: 90','Not: 60']
console.log(ortalama);   // 62
```

12. Sınav tarzı sorular — cevaplarıyla

S1. `const x = [2, 3]; x.push(4);` hata verir mi? **Cevap:** Hayır; değişkenin işaret ettiği dizi değiştirilir.

S2. `console.log(0 || 15, 0 ?? 15)` ne yazar? **Cevap:** `15 0`.

S3. `['a','b','c'].map((v,i) => i + v)` sonucu? **Cevap:** `['0a','1b','2c']`.

S4. `5 == '5'` ve `5 === '5'` sonucu? **Cevap:** `true`, `false`.

S5. Bir listeden yalnızca 50 üstü notları almak için hangi metot? **Cevap:** `filter(n => n > 50)`.

S6. `const { ad } = kullanici` ne yapar? **Cevap:** `kullanici.ad` özelliğini `ad` adlı yerel değişkene ayırır.

S7. `for...of` ile `for...in` farkı? **Cevap:** İlki değer, ikincisi özellik anahtarı üretir.

S8. `import { topla }` ve `import topla` farkı? **Cevap:** İlki named, ikincisi default export içindir.

13. Sınavda hızlı bakış

İhtiyaç	Hazır kalıp
Değişken	`const ad='Ünal'; let sayac=0;`
Koşul	`const yazi = n>=50 ? 'Geçti' : 'Kaldı';`
Döngü	`for (const x of dizi) { console.log(x); }`
Diziye yeni öğe (değiştirmeden)	`const yeni = [...eski, eleman];`
Elemanı sil (yeni dizi)	`const yeni = dizi.filter(x => x.id !== id);`
Özellik güncelle	`const yeni = {...kisi, ad:'Ali'};`
Sayı toplamı	`dizi.reduce((t,x) => t+x, 0)`
Varsayılan değer	`deger ?? varsayilan`

Kaynak dosyalar: W2/README.md; W2/topics.
Bu çalışma rehberi ders deposunun eğitim konularına göre özgün Türkçe özet olarak hazırlanmıştır; hocanın birebir sınav soruları değildir.

W3 — React ve JSX | Sınav Çalışma Dosyası

Kaynak: zeki-ozen/MobileAppDevelopment / W3
Kapsam: Bileşenler, JSX, props, state, yaşam döngüsü, events, koşul/listeler, hooks, veri paylaşımı, stiller, formlar ve React Router.
DİKKAT: W3 deposundaki örneklerin büyük kısmı **web React** içindir (`div`, `input`, CSS, React Router). Bunları Expo Snack'e aynen yapıştırmak doğru değildir. Mobil karşılıkları W4/W6 dosyalarında verildi.

1. React nedir? Bileşen mantığı

React, kullanıcı arayüzlerini **bileşen** (component) adı verilen tekrar kullanılabilir parçalardan oluşturan bir JavaScript kütüphanesidir. UI, state ve props değişimlerine göre yeniden hesaplanır. Bileşen adları **büyük harfle** başlar (`MerhabaKart`). Modern projelerde fonksiyon bileşenleri ve Hook'lar en yaygın yaklaşımdır.

```
function Karsilama({ ad }) {
  return <h2>Merhaba {ad}!</h2>;
}
export default function App() {
  return <Karsilama ad="Ünal" />;
}
```

Kural: `<Karsilama />` bir React bileşenidir; `<h2>` bir HTML elementidir. `function Karsilama()` kodu JSX döndürür. `export default` başka dosyalardan bu bileşeni import etmeyi sağlar.

Fonksiyon ve sınıf bileşeni

```
// Fonksiyon bileşeni
function Baslik({ metin }) { return <h1>{metin}</h1>; }

// Eski stil: class bileşeni
class Selam extends React.Component {
  render() { return <h1>Selam {this.props.ad}</h1>; }
}
```

Sınıf bileşenleri `this.state`, `setState`, `componentDidMount` gibi API'ler kullanır. Modern fonksiyon bileşenleri `useState`, `useEffect` kullanır.

2. JSX kuralları

JSX, JavaScript içinde HTML'ye benzeyen UI yazma biçimidir. JSX ifadesinde JavaScript hesaplamaları `{ ... }` içine yazılır. Önemli kurallar:

1. Tek bir kök element döndür: `<div>...</div>` veya `<>...</>`.
2. HTML tarafında `class` yerine `className`, `for` yerine `htmlFor` kullan.
3. Açık/kapalı etiketleri eşleştir; boş etiketleri `<img />` şeklinde yaz.
4. `onClick` olayına **fonksiyon** ver, `onClick={tikla}`; `onClick={tikla()}` yazarsan render sırasında çağırırsın.

```
const ad = 'Ünal';
const puan = 70;
const ekran = (
  <div className="kart">
    <h3>{ad.toUpperCase()}</h3>
    <p>Sonuç: {puan >= 50 ? 'Geçti' : 'Kaldı'}</p>
    <img src="/logo.png" alt="Logo" />
  </div>
);
```

Mobil ayrımı: React Native'de `<div>`, `<h3>`, `<p>`, `<img>` **yoktur**; genellikle `<View>`, `<Text>`, `<Image>` kullanılır. `className` yerine çoğunlukla `style` kullanılır.

3. Props (bileşenler arası veri aktarma)

Props = üst bileşenden alt bileşene gönderilen ve alt bileşenin değiştirmemesi gereken girdiler. Props, fonksiyonun parametresi gibidir. `children`, etiketin arasındaki içeriği taşır.

```
function Kart({ baslik = 'Başlıksız', children }) {
  return (
    <section>
      <h3>{baslik}</h3>
      <div>{children}</div>
    </section>
  );
}
function App() {
  return <Kart baslik="Sınav"><p>Çalışmaya devam!</p></Kart>;
}
```

`baslik = 'Başlıksız'` varsayılan parametre. `children` içine gönderilen JSX. Alt bileşen `props.baslik = ...` yapmamalı. Değişiklik gerekiyorsa parent state'ini callback ile güncelle.

4. State ve `useState`

State = bileşenin değişebilen belleği. Değiştiğinde React yeni render yapar. `useState(başlangıç)` → `[değer, güncellemeFonksiyonu]`.

```
import { useState } from 'react';

export default function Sayac() {
  const [sayac, setSayac] = useState(0);
  return (
    <div>
      <p>Sayac: {sayac}</p>
      <button onClick={() => setSayac(s => s + 1)}>+1</button>
    </div>
  );
}
```

Neden `setSayac(s => s + 1)`? Güncelleme eski değere bağlıysa fonksiyonel biçim bekleyen güncellemelerle güvenilir çalışır.

```
// YANLIŞ: state doğrudan mutasyona uğratılır.
// kullanicilar.push(yeniKullanici);
// setKullanicilar(kullanicilar);

// DOĞRU: yeni dizi üret.
setKullanicilar(eski => [...eski, yeniKullanici]);
setKullanicilar(eski => eski.filter(k => k.id !== silinecekId));
```

Props	State
Parent'tan gelir	Bileşenin kendisi yönetir
Alt bileşen açısından salt okunur	Setter ile değiştirilir
Örn. `<Kart isim="Ünal" />`	Örn. `[isim, setIsim] = useState('Ünal')`

5. Etkinlik yönetimi (Events)

```
function Dugme() {
  function tiklandi() { alert('Butona basıldı'); }
  return <button onClick={tiklandi}>Bas</button>;
}
```

React web: `onClick`, `onChange={e => ...}`, `e.target.value`, `onSubmit`, `e.preventDefault()`. **React Native:** tıklama için `onPress`, metin girişi için `onChangeText` kullanılır; tarayıcı `event.target.value` kalıbı doğrudan Expo'da aynı değildir.

6. Koşullu gösterim ve liste

```
function Durum({ girisYapti }) {
  if (!girisYapti) return <p>Lütfen giriş yap</p>;
  return <p>Hoş geldin!</p>;
}

function Rozet({ puan }) {
  return <strong>{puan >= 50 ? 'Geçti' : 'Kaldı'}</strong>;
}

function Liste({ kisiler }) {
  return <ul>{kisiler.map(k => <li key={k.id}>{k.ad}</li>)}</ul>;
}
```

- `&&` ile gösterim: `{hazir && <p>Hazır</p>}`.
- **Tuzak:** `{sayi && <p>Var</p>}` ifadesi `sayi = 0` ise ekranda `0` oluşturabilir; `{sayi > 0 && ...}` daha güvenlidir.
- `key` benzersiz ve **kararlı** olmalıdır: verinin `id` alanı idealdir. Değişen listelerde dizin (`index`) key olarak sorun çıkarabilir.
- React'te `map` UI elemanları üretir; `filter` görünmesini istemediklerini çıkarır.

7. `useEffect`: yan etki ve yaşam döngüsü

Yan etki (side effect): Ağ isteği, abonelik, zamanlayıcı, harici API gibi render dışı işler. `useEffect` bu işler için kullanılır.

```
import { useEffect, useState } from 'react';
function Saat() {
  const [saniye, setSaniye] = useState(0);
  useEffect(() => {
    const id = setInterval(() => setSaniye(s => s + 1), 1000);
    return () => clearInterval(id); // temizleme
  }, []);
  return <p>{saniye} saniye</p>;
}
```

Biçim	Genel davranış
`useEffect(fn)`	Her commit sonrası çalışabilir
`useEffect(fn, [])`	Bileşen mount olduğunda; development Strict Mode'da ek test çalışmaları olabilir
`useEffect(fn, [id])`	Mount ve `id` değiştiğinde çalışır
`return () => ...`	Temizleme: bağımlılık değişmeden önce/unmount sırasında

Önemli: Hook'u `if` içinde veya döngüde çağırma. Hook çağrı sırası her render'da aynı kalmalıdır. `useEffect(async () => ...)` yerine effect içinde async fonksiyon tanımlayıp çağırmak daha uygundur.

8. Diğer önemli Hook'lar

Hook	Ne işe yarar?	Kısa örnek
`useState`	Yerel değişen değer	`const [x,setX] = useState(0)`
`useEffect`	Yan etki / abonelik	`useEffect(() => {...}, [])`
`useRef`	Render tetiklemeden kalıcı referans	`const ref = useRef(null)`
`useContext`	Uzak bileşene ortak veri	`useContext(ThemeContext)`
`useReducer`	Karmaşık state geçişleri	`useReducer(reducer, initial)`
`useMemo`	Hesaplanan değeri önbellekleme	`useMemo(() => hesapla(x), [x])`
`useCallback`	Fonksiyon referansını koruma	`useCallback(() => kaydet(id), [id])`

`useMemo` ve `useCallback` çoğu küçük projede şart değildir; performans sorunu varsa anlamlıdır. `useRef.current` güncellenmesi tek başına render tetiklemez.

```
const [adet, setAdet] = useState(1);
const ikiKat = useMemo(() => adet * 2, [adet]);
const arttir = useCallback(() => setAdet(x => x + 1), []);
```

9. Bileşenler arasında veri paylaşımı

En sık çözüm **lifting state up**: Ortak veriyi iki alt bileşenin **en yakın ortak üst bileşeninde** tut ve aşağı props/callback gönder.

```
function Goster({ deger }) { return <p>{deger}</p>; }
function Arttir({ onArttir }) {
  return <button onClick={onArttir}>Artır</button>;
}
function App() {
  const [n, setN] = useState(0);
  return <><Goster deger={n} /><Arttir onArttir={() => setN(x => x + 1)} /></>;
}
```

Prop drilling: Çok sayıda katmandan props geçirme. Çok yaygın ve ortak veriler için Context API, büyük ölçekli ihtiyaçlar için ek state kütüphaneleri kullanılabilir.

10. Web React stilleri ve CSS

```
const kartStili = { backgroundColor: '#eef', padding: 16 };
function Kart() {
  return <div className="kart" style={kartStili}>İçerik</div>;
}
```

React web'de haricî `.css` dosyası, CSS Modules, CSS-in-JS, Tailwind kullanılabilir. `style` **JavaScript nesnesi** alır; `background-color` yerine `backgroundColor` yazılır. React Native'de ayrıca `StyleSheet.create()` vardır ve CSS dosyası React Native çekirdek bileşenlerine doğrudan uygulanmaz.

11. Formlar ve kontrollü bileşen

Kontrollü formda `input` değerinin kaynağı React state'idir.

```
import { useState } from 'react';
function AdFormu() {
  const [ad, setAd] = useState('');
  const [hata, setHata] = useState('');

  function gonder(e) {
    e.preventDefault();
    setHata(ad.trim() ? '' : 'Ad boş bırakılamaz');
    if (ad.trim()) alert('Merhaba ' + ad);
  }
  return (
    <form onSubmit={gonder}>
      <input value={ad} onChange={e => setAd(e.target.value)} />
      {hata && <p>{hata}</p>}
      <button type="submit">Gönder</button>
    </form>
  );
}
```

Kontrollü: `value={state}` ve `onChange` birlikte. **Kontrolsüz:** Değer DOM üzerinde tutulur (`ref`). React Native'de benzer kontrollü kalıp `<TextInput value={ad} onChangeText={setAd} />` biçimindedir.

12. Web yönlendirmesi: React Router

React Router, URL değiştiğinde başka sayfa bileşeni göstermeye yarar. **W3 konusu web React içindir**; Expo uygulamasında genellikle Expo Router veya React Navigation kullanılır.

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';
function AnaSayfa() { return <h2>Ana sayfa</h2>; }
function Hakkinda() { return <h2>Hakkında</h2>; }
function App() {
  return (
    <BrowserRouter>
      <nav><Link to="/">Ana</Link> | <Link to="/hakkinda">Hakkında</Link></nav>
      <Routes>
        <Route path="/" element={<AnaSayfa />} />
        <Route path="/hakkinda" element={<Hakkinda />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Not: Bu örnek W3'teki React Router v6 tarzını gösterir. Paket sürümüne göre bazı React Router API'leri farklı olabilir.

13. Mini proje — Web React görev listesi (tam örnek)

Bu kod **React web** projesinde `App.jsx` içine uygundur. **Expo Snack'e doğrudan yapıştırma:** `<div>`, `<input>`, `<button>` gibi web etiketleri içerir.

```
import { useState } from 'react';
export default function App() {
  const [yeni, setYeni] = useState('');
  const [gorevler, setGorevler] = useState([
    { id: 1, ad: 'JS tekrar et' },
  ]);
  function ekle() {
    if (!yeni.trim()) return;
    setGorevler(eski => [...eski, { id: Date.now(), ad: yeni.trim() }]);
    setYeni('');
  }
  return (
    <div>
      <h1>Görev Listem</h1>
      <input value={yeni} onChange={e => setYeni(e.target.value)} />
      <button onClick={ekle}>Ekle</button>
      <ul>
        {gorevler.map(g => (
          <li key={g.id}>
            {g.ad}
            <button onClick={() => setGorevler(eski => eski.filter(x => x.id !== g.id))}>
              Sil
            </button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

14. W3 React web → Expo React Native hızlı dönüşüm

W3 (Web React)	W4/W6 (React Native)
`<div>`	`<View>`
`<p>`, `<span>`, `<h1>`	`<Text>` + `style`
`<button onClick={fn}>`	`<Button title="Bas" onPress={fn} />`
`<input onChange={e => ...}>`	`<TextInput onChangeText={...} />`
`<img src="..." />`	`<Image source={{ uri: '...' }} style={{width:...,height:...}} />`
CSS `.kart { ... }`	`StyleSheet.create({ kart: { ... } })`
`react-router-dom`	Expo Router / React Navigation
`useState`, `useEffect`	Aynı Hook'lar kullanılır

15. Sınav soruları ve kısa cevaplar

S1. `props` ve `state` farkı nedir? **Cevap:** Props dışarıdan gelen salt-okunur girdiler; state bileşenin güncellenen yerel verisidir.

S2. `onClick={arttir()}` neden sorunlu? **Cevap:** Fonksiyonu render sırasında çağırır; olay işleyicisi olarak fonksiyon referansı verilmeli (`onClick={arttir}`).

S3. `useEffect(..., [])` hangi duruma karşılık gelir? **Cevap:** Mount ile ilişkili effect; temizleme unmount'ta, geliştirme Strict Mode özel davranabilir.

S4. `key` neden gerekli? **Cevap:** React'in listedeki öğelerin kimliğini koruyup güncellemeleri doğru ilişkilendirmesi için.

S5. `setListe([...liste, yeni])` ile `liste.push(yeni)` farkı? **Cevap:** İlki yeni dizi oluşturur ve React state güncellemesine uygundur; ikincisi mevcut diziyi değiştirir.

S6. `useRef` güncellemesi her zaman yeniden render yapar mı? **Cevap:** Hayır.

S7. `children` nedir? **Cevap:** Bileşen etiketlerinin arasındaki JSX içeriğidir.

S8. Web React kodunda `<button>` yerine Expo'da hangi bileşen? **Cevap:** `Button`, `Pressable` veya `TouchableOpacity`.

16. Hızlı başvuru

Sorun	Kalıp
Bileşen oluştur	`function Kart({ad}) { return <p>{ad}</p> }`
State oluştur	`const [x, setX] = useState(0);`
Eski state'e göre artır	`setX(v => v + 1);`
Koşullu render	`{hazir ? <A /> : <B />}`
Liste render	`{liste.map(x => <li key={x.id}>{x.ad}</li>)}`
Effect temizliği	`useEffect(() => { const id=...; return () => clearInterval(id); }, [])`
Parent'tan callback	`<Buton onArttir={() => setX(v => v+1)} />`

Kaynak dosyalar: W3/README.md, W3 bileşen ve JSX notları.
Bu dosya, kaynak depoya dayanan Türkçe sınav rehberidir; web ve mobil kodların farkları özellikle vurgulanmıştır.

W4 — React Native ve Expo | Sınav Çalışma Dosyası

Kaynak: zeki-ozen/MobileAppDevelopment / W4
Kapsam: React Native girişi, Expo/Hello Expo, `let` / `const`, props, state, Hook'lar (`useState`, `useEffect`).
Öncelik: Kod parçalarının çoğu **Expo Snack** `App.js` içinde kullanılabilir. Tam uygulama örneklerini `App.js` içine **tek tek** yapıştır.

1. React Native ve Expo nedir?

React Native: JavaScript/React ile Android ve iOS'ta yerel arayüz bileşenleri kullanarak mobil uygulama geliştirme teknolojisi. **Expo:** React Native geliştirmeyi kolaylaştıran araçlar ve servisler; Expo Go, desteklenen projeleri telefonda test etmeye yarar. **Expo Snack:** Tarayıcıda React Native kodu yazıp önizleyebileceğin internet editörü.

Özellik	React (web)	React Native (mobil)
Ekran bileşeni	`<div>`, `<p>`, `<button>`	`<View>`, `<Text>`, `<Button>`
Stil	CSS, CSS Modules	JS stil nesnesi / `StyleSheet`
Tıklama	`onClick`	`onPress`
Metin kutusu	`<input>`	`<TextInput>`
Yerel donanım	Tarayıcı API'leri	Uygun React Native/Expo modülleri

> **W4 sınav ipucu:** `useState` ve `useEffect` mantığı React web ile aynıdır; JSX'te kullanılan **ekran bileşenleri farklıdır**.

2. Expo Snack web editöründe başlama (Windows + iPhone)

1. Bilgisayarda snack.expo.dev aç.
2. `App.js` dosyasını seç; örnek uygulamanın kodunu buraya yapıştır.
3. Gerekli paket varsa Snack `Dependencies` bölümünden ekle.
4. `My Device` görünümündeki QR kodunu iPhone kameranla tara; **Expo Go** ile aç.
5. Kod değiştirip kaydettikçe uygulama önizlemesini güncelle.

Yerel proje (alternatif): Node.js kurulu bilgisayarda `npx create-expo-app@latest` ile proje oluşturup `npx expo start` çalıştırılır. Şablona bağlı olarak ana dosya `App.js` değil `app/index.tsx` veya `src/app/index.tsx` olabilir. **Snack'te ise öğretimde genellikle `App.js` kullanılır.** Windows'ta iOS simulator yerel olarak kullanılamaz, ancak iPhone + Expo Go ile test edebilirsin.

3. En temel `App.js` iskeleti — TAM VE ÇALIŞIR

```
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

export default function App() {
  return (
    <View style={styles.ekran}>
      <Text style={styles.baslik}>Merhaba Expo!</Text>
      <Text>İlk mobil uygulamam.</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  ekran: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#F1F5F9',
  },
  baslik: { fontSize: 26, fontWeight: 'bold', color: '#0F172A' },
});
```

Satırların görevi: `import` gerekli bileşenleri getirir. `App` ana bileşendir. `return (...)` ekranda görünecek JSX'i belirtir. `View` kutudur, `Text` yazıdır. `styles.ekran` stil kurallarını uygular. `export default` uygulamanın ana bileşenini dışa açar.

4. `View`, `Text`, `StyleSheet` ve Flexbox

Stil	Ne yapar?	Sık kullanılan değer
`flex: 1`	Mevcut kullanılabilir alanı doldurur	`1`
`flexDirection`	Çocukların yönü	`'column'` (RN varsayılanı), `'row'`
`justifyContent`	Ana eksende hizalama	`'center'`, `'space-between'`
`alignItems`	Çapraz eksende hizalama	`'center'`, `'stretch'`
`padding` / `margin`	İç/dış boşluk	`12`, `20`
`backgroundColor`	Arka plan	`'#ffffff'`
`borderRadius`	Yuvarlatma	`12`
`fontSize`	Yazı boyutu (`Text`)	`18`

```
<View style={{ flexDirection: 'row', justifyContent: 'space-between' }}>
  <Text>Sol</Text><Text>Sağ</Text>
</View>
```

Tuzak: RN'de `width: '100%'` veya `width: 120` kullan; `width: '120px'` web CSS biçimidir. `Text` dışındaki düz yazılar React Native'de doğrudan çocuk metin olarak gösterilmez; `<Text>` içine yazılmalı.

5. `const` / `let` ve render değişkenleri

```
const baslik = 'Mobil Programlama';  // bağın yeniden atanması yasak
let puan = 5;                        // yeniden atanabilir
puan = puan + 2;
const renkler = ['mavi', 'mor'];
renkler.push('yeşil');               // const dizinin içeriği değişebilir
```

Çok önemli: React bileşeninin içindeki normal `let sayac = 0` değerini artırmak **ekranı güncellemez**. Kalıcı ve görünür şekilde değişmesi için `useState` kullan.

6. Props — Parent'tan çocuğa veri aktarımı

Aşağıdaki tam örnekte `OgrenciKart` bileşenini aynı ekranda farklı verilerle tekrar kullanıyoruz.

```
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

function OgrenciKart({ ad, not }) {
  return (
    <View style={styles.kart}>
      <Text style={styles.ad}>{ad}</Text>
      <Text>{not >= 50 ? 'Geçti' : 'Kaldı'} ({not})</Text>
    </View>
  );
}

export default function App() {
  return (
    <View style={styles.ekran}>
      <OgrenciKart ad="Ünal" not={85} />
      <OgrenciKart ad="Ayşe" not={45} />
    </View>
  );
}

const styles = StyleSheet.create({
  ekran: { flex: 1, justifyContent: 'center', padding: 24, gap: 14 },
  kart: { padding: 18, backgroundColor: '#DBEAFE', borderRadius: 12 },
  ad: { fontSize: 20, fontWeight: 'bold' },
});
```

`ad="Ünal"` bir string prop'tur; `not={85}` JavaScript sayısıdır. `props` alt bileşende doğrudan değiştirilmez. Değiştirmeyi üst bileşende state ve callback ile yaparsın.

7. State + buton + sayaç — TAM `App.js`

```
import React, { useState } from 'react';
import { View, Text, Button, StyleSheet } from 'react-native';

export default function App() {
  const [sayi, setSayi] = useState(0);
  return (
    <View style={styles.ekran}>
      <Text style={styles.baslik}>Sayaç: {sayi}</Text>
      <Button title="+1 artır" onPress={() => setSayi(x => x + 1)} />
      <Button title="-1 azalt" onPress={() => setSayi(x => x - 1)} />
      <Button title="Sıfırla" onPress={() => setSayi(0)} />
    </View>
  );
}

const styles = StyleSheet.create({
  ekran: { flex: 1, justifyContent: 'center', padding: 30, gap: 16 },
  baslik: { fontSize: 28, fontWeight: 'bold', textAlign: 'center' },
});
```

Sınavda açıklaması: `useState(0)` ilk değeri sıfır yapar. `sayi` okunacak değerdir. `setSayi` günceller. `onPress` butona dokunulunca fonksiyon çağırır. `setSayi(x => x + 1)` önceki state'e göre güvenli artırma sağlar.

8. TextInput ile kontrollü form — TAM `App.js`

```
import React, { useState } from 'react';
import { View, Text, TextInput, Button, Alert, StyleSheet } from 'react-native';

export default function App() {
  const [ad, setAd] = useState('');
  const [yas, setYas] = useState('');

  function gonder() {
    if (!ad.trim() || !yas.trim()) {
      Alert.alert('Uyarı', 'İsim ve yaş zorunludur.');
      return;
    }
    Alert.alert('Kayıt', `${ad}, ${yas} yaşındasın.`);
  }

  return (
    <View style={styles.ekran}>
      <Text style={styles.baslik}>Kayıt Formu</Text>
      <TextInput
        style={styles.girdi}
        placeholder="Adını yaz"
        value={ad}
        onChangeText={setAd}
      />
      <TextInput
        style={styles.girdi}
        placeholder="Yaşını yaz"
        keyboardType="number-pad"
        value={yas}
        onChangeText={setYas}
      />
      <Button title="Kaydet" onPress={gonder} />
    </View>
  );
}

const styles = StyleSheet.create({
  ekran: { flex: 1, justifyContent: 'center', padding: 24, gap: 15 },
  baslik: { fontSize: 24, fontWeight: 'bold' },
  girdi: { borderWidth: 1, borderColor: '#94A3B8', borderRadius: 10, padding: 12 },
});
```

`TextInput` her zaman metin (`string`) döndürür; sayısal klavye `keyboardType="number-pad"` olsa da sayıya dönüştürme gerekebilir: `Number(yas)`. `trim()` boşluk girilmesini boş kabul eder. `Alert.alert()` iPhone'da da uyarı penceresi gösterir.

9. `useEffect` ve zamanlayıcı — TAM `App.js`

```
import React, { useEffect, useState } from 'react';
import { View, Text, Button } from 'react-native';

export default function App() {
  const [saniye, setSaniye] = useState(0);
  const [calisiyor, setCalisiyor] = useState(false);

  useEffect(() => {
    if (!calisiyor) return;
    const id = setInterval(() => setSaniye(x => x + 1), 1000);
    return () => clearInterval(id);
  }, [calisiyor]);

  return (
    <View style={{ flex: 1, justifyContent: 'center', padding: 30, gap: 12 }}>
      <Text style={{ fontSize: 26 }}>Geçen süre: {saniye} sn</Text>
      <Button title={calisiyor ? 'Durdur' : 'Başlat'}
              onPress={() => setCalisiyor(x => !x)} />
      <Button title="Sıfırla" onPress={() => setSaniye(0)} />
    </View>
  );
}
```

Satır mantığı: `useEffect` `calisiyor` değişince çalışır. Açıkken 1 saniyelik timer başlar. Return içindeki `clearInterval` eski timer'ı temizler; timer birikmesini önler. Hook'u koşul içinde değil, effect'in içindeki koşulla kullanıyoruz.

10. Sık karşılaşacağın kod kalıpları

İstediğin şey	Kopyalanacak kalıp
Ekrana yazı koy	`<Text>Merhaba</Text>`
Bir kutu oluştur	`<View style={{padding: 20}}>` … `</View>`
Buton ekle	`<Button title="Bas" onPress={() => Alert.alert('Basıldı')} />`
State tanımla	`const [ad, setAd] = useState('');`
Metin al	`<TextInput value={ad} onChangeText={setAd} />`
Şarta göre göster	`{gorunur && <Text>Görünüyorum</Text>}`
İki yazıyı yan yana koy	`flexDirection: 'row'`
Ekranı ortala	`flex:1, justifyContent:'center', alignItems:'center'`
Listeye yeni kayıt ekle	`setListe(eski => [...eski, yeni]);`
Koşula göre renk	`backgroundColor: aktif ? 'green' : 'gray'`

11. Yaygın hatalar ve çözümleri

Hata	Neden / çözüm
`Text strings must be rendered within a <Text>`	Düz yazıyı `<Text>` içine al
`Can't find variable: useState`	`import { useState } from 'react'` unutulmuş
Buton çalışmıyor	`onPress={() => islem()}` veya `onPress={islem}` kullan
Yazı görünmüyor	`Text` yerine `View` içine düz metin yazılmış olabilir
State değişiyor ama ekran değişmiyor	Doğrudan `state = ...` yerine setter kullan
Foto görünmüyor	`Image` için genişlik/yükseklik veya doğru `source` gerekli
`className` beklediğin gibi çalışmıyor	Saf React Native'de `style` / `StyleSheet` kullan
`.map` liste elemanları hata veriyor	Döndürülen öğelere kararlı `key` ekle

12. W4 öz değerlendirme soruları

S1. Expo Snack ile Expo Go farkı? **Cevap:** Snack tarayıcıda kod editörüdür; Expo Go desteklenen projeyi telefonda açıp test etmeye yarar.

S2. React web'deki `<div>` mobilde ne olur? **Cevap:** `<View>`.

S3. `const [sayi, setSayi] = useState(0)` içinde `sayi` ve `setSayi` nedir? **Cevap:** State değeri ve setter.

S4. `onPress={fonksiyon()}` neden istenmeyebilir? **Cevap:** Fonksiyonu render anında çalıştırabilir; olay sırasında çalışması için `onPress={fonksiyon}` ver.

S5. `useEffect` cleanup niçin önemli? **Cevap:** Timer, listener ve diğer kaynakları temizlemek, tekrarlandıklarında çoğalmalarını önlemek için.

S6. `props` alt bileşen tarafından doğrudan değiştirilmeli mi? **Cevap:** Hayır.

S7. Flexbox'ta React Native varsayılan yön? **Cevap:** `column`.

S8. Sayısal klavyeli TextInput'un değeri number mıdır? **Cevap:** Hayır, state'te string tutulur; gerekiyorsa dönüşüm yapılır.

13. Depoyla eşleştirme

Depo dosyası	Bu rehberde
`1-react-native-intro.md`	1–4, 11
`2-example-hello-expo.md`	2–3
`3-react-native-variable-const.md`	5
`4-react-native-props.md`	6
`5-react-native-state.md`	7–8

Depo dosyası	Bu rehberde
`6-react-native-hooks.md`	9

Kaynak: W4 klasörü.
Bu çalışma dosyası eğitim amaçlı özgün Türkçe özet ve örnek kodlardan oluşur; Expo SDK sürümü ve şablon değişebileceği için mevcut proje dosya yapını kontrol et.

W6 — React Native Temel Bileşenler | Sınav Çalışma Dosyası

Kaynak: zeki-ozen/MobileAppDevelopment / W6
Kapsam: `View`, `Text`, `Image`, `TextInput`, `Button`, `ScrollView`, `FlatList`, `TouchableOpacity`, `SafeAreaView`, `Picker`, `RadioButton`, `Checkbox`.
Hedef: Hangi bileşen ne işe yarar, hangi prop ile kullanılır, sık yapılan hatalar ve **Expo Snack** `App.js` örnekleri.

> **Kaynak notu:** `Picker` ve `Checkbox` için ayrı paket gerekir. `RadioButton` çekirdek React Native'de hazır bir bileşen değildir; bu rehberde `Pressable` ile yapılır. `SafeAreaView` için modern projelerde `react-native-safe-area-context` tercih edilir. Bu farklar ders deposundaki W6 anlatımlarıyla uyumludur.

1. Bileşenleri 30 saniyede tanı

Bileşen	Görevi	En önemli prop/olay
`View`	Kutu, satır, kolon, düzen	`style`
`Text`	Yazı gösterme	`style`, `numberOfLines`
`Button`	Hızlı hazır buton	`title`, `onPress`, `disabled`
`TouchableOpacity`	Özelleştirilebilir basılabilir alan	`onPress`, `activeOpacity`
`Pressable`	Esnek basılma davranışı	`onPress`, `onPressIn`, `style`
`Image`	Yerel veya internet görseli	`source`, `style`, `resizeMode`
`TextInput`	Kullanıcıdan metin alma	`value`, `onChangeText`, `placeholder`
`ScrollView`	İçeriği kaydırma	`horizontal`, `contentContainerStyle`
`FlatList`	Çok öğeli, performanslı liste	`data`, `renderItem`, `keyExtractor`
`SafeAreaView`	Çentik/status bar boşluğu	`edges`, `style` (safe-area-context)
`Picker`	Bir seçenek seçme	`selectedValue`, `onValueChange`
`RadioButton`	Tek seçenekli özel kontrol	`Pressable` ile seçili state
`Checkbox`	Aç/kapat, birden çok işaret	`value`, `onValueChange`

2. `View` — yerleşim kutusu

`View` bileşenleri gruplayan ana kapsayıcıdır. `div` karşılığına yakındır ama React Native **Flexbox** ile yerleşim yapar.

```
<View style={{ flex: 1, padding: 20, backgroundColor: '#F8FAFC' }}>
  <Text>İçerik</Text>
</View>
```

`flexDirection: 'row'` yatay, `'column'` dikey yerleşim. `justifyContent` ana eksen, `alignItems` çapraz eksen hizalamasıdır. `View` içine doğrudan `Merhaba` yazısı koyma; `<Text>Merhaba</Text>` kullan.

3. `Text` — metin, satır kısıtlama

```
<Text style={{ fontSize: 24, fontWeight: 'bold', color: '#1D4ED8' }}>
  Hoş geldin ÜNAL
</Text>
<Text numberOfLines={1} ellipsizeMode="tail">
  Çok uzun bir açıklama gerektiğinde sonunu kısaltabiliriz.
</Text>
```

`numberOfLines={1}` tek satıra sınırlar. `ellipsizeMode="tail"` gerekirse sonunu `...` ile kısaltır. İç içe `Text` ile farklı renk/kalınlıkta metin hazırlanabilir.

4. `Button`, `TouchableOpacity`, `Pressable` — tıklamalar

Hazır `Button`: Kullanması en kolay, stil seçenekleri kısıtlıdır.

```
<Button title="Göster" onPress={() => Alert.alert('Merhaba!')} />
<Button title="Pasif" onPress={() => {}} disabled />
```

Özel buton `TouchableOpacity`: İçine `Text`, `Image`, `View` koyabilirsin.

```
<TouchableOpacity
  activeOpacity={0.7}
  onPress={() => Alert.alert('Butona basıldı')}
  style={{ backgroundColor: '#2563EB', padding: 16, borderRadius: 12 }}
>
  <Text style={{ color: 'white', textAlign: 'center' }}>Özel Buton</Text>
</TouchableOpacity>
```

`Pressable`: Yeni etkileşimlerde tercih edilebilir. `pressed` durumuna göre stil değiştirebilir.

```
<Pressable
  onPress={() => console.log('Basılma tamamlandı')}
  style={({ pressed }) => ({
    padding: 14,
    borderRadius: 10,
    backgroundColor: pressed ? '#1D4ED8' : '#60A5FA',
  })}
>
  <Text style={{ color: '#fff' }}>Bana bas</Text>
</Pressable>
```

Tuzak: `onPress={() => islem()}` doğrudur; `onPress={islem()}` fonksiyon sonucu verir ve erken çalıştırabilir.

5. `TextInput` — kontrollü veri girişi

```
const [ad, setAd] = useState('');
<TextInput
  value={ad}
  onChangeText={setAd}
  placeholder="Adını yaz"
  autoCapitalize="words"
  style={{ borderWidth: 1, borderColor: '#CBD5E1', padding: 12 }}
/>
<Text>Girilen: {ad}</Text>
```

Prop	İşlevi
`placeholder`	Boşken görünen ipucu
`value`	State'te tutulan değer
`onChangeText`	Metin her değiştiğinde günceller
`keyboardType="email-address"`	E-posta klavyesi
`keyboardType="numeric"`	Sayısal klavye (değer yine string)
`secureTextEntry`	Şifreyi gizler
`multiline`	Çok satırlı giriş
`maxLength={20}`	En fazla 20 karakter

Önemli: `TextInput` için web'deki `onChange={e => setAd(e.target.value)}` kalıbı yerine genellikle **`onChangeText={setAd}`** kullanılır.

6. `Image` — resim ekleme

```
<Image
  source={{ uri: 'https://reactnative.dev/img/tiny_logo.png' }}
  style={{ width: 120, height: 120, borderRadius: 12 }}
  resizeMode="contain"
/>
```

Yerel resim: `source={require('./assets/logo.png')}` (resmin dosyada mevcut olması şart). İnternet görseli için `source={{ uri: 'https://...' }}`. `resizeMode`: `cover` alanı doldurup kırpabilir, `contain` tamamını gösterir. Özellikle uzak resimde `width` ve `height` belirlemek gerekir.

7. `ScrollView` — kısa, kaydırılabilir içerik

```
<ScrollView contentContainerStyle={{ padding: 20, gap: 10 }}>
  <Text>Üst yazı</Text>
  <Text>Çok sayıda içerik...</Text>
</ScrollView>
```

Kısa formlar ve bilgi sayfaları için uygundur. İçindeki **tüm** çocukları render eder; binlerce kayıt varsa performans için `FlatList` kullan. `horizontal` yatay kaydırma sağlar.

8. `FlatList` — dizi listeleme

```
const urunler = [
  { id: 'a', ad: 'Kalem' },
  { id: 'b', ad: 'Defter' },
];
<FlatList
  data={urunler}
  keyExtractor={item => item.id}
  renderItem={({ item }) => (
    <Text style={{ padding: 12 }}>{item.ad}</Text>
  )}
/>
```

Prop açıklaması: `data` listedeki dizi; `renderItem` her öğenin görünümü; `keyExtractor` benzersiz kimlik; `ListEmptyComponent` liste boşken gösterim; `ItemSeparatorComponent` ayırıcı; `horizontal` yatay liste; `onEndReached` sonlara yaklaşıldığında olay; `refreshing` / `onRefresh` yenileme davranışı.

Sınavda kritik ayrım: `ScrollView` tüm çocukları birden oluşturur. `FlatList` sanallaştırma (virtualization) ile uzun listelerde daha verimlidir. `FlatList` bir öğeye otomatik `key` bulmaya çalışabilir; açıkça `keyExtractor` vermek daha sağlamdır.

9. `SafeAreaView` — iPhone çentiği ve güvenli alan

Modern kullanım için Snack Dependencies içine **`react-native-safe-area-context`** ekle (Snack'te kurulu olabilir):

```
import { SafeAreaProvider, SafeAreaView } from 'react-native-safe-area-context';

export default function App() {
  return (
    <SafeAreaProvider>
      <SafeAreaView edges={['top', 'bottom']} style={{ flex: 1 }}>
        <Text>Çentikten ve alt göstergeden uzak içerik</Text>
      </SafeAreaView>
    </SafeAreaProvider>
  );
}
```

Not: `SafeAreaView` ismi geçmişte `react-native` içinde de yer alır; güncel projelerde `react-native-safe-area-context` çözümü tercih edilir. `SafeAreaProvider` çoğu tam projede kök uygulama düzeyinde bir kez tanımlanır.

10. `Picker` — seçmeli menü (ek paket)

Snack **Dependencies** içine `@react-native-picker/picker` ekle:

```
import React, { useState } from 'react';
import { View, Text } from 'react-native';
import { Picker } from '@react-native-picker/picker';

export default function App() {
  const [sehir, setSehir] = useState('istanbul');
  return (
    <View style={{ flex: 1, justifyContent: 'center', padding: 20 }}>
      <Text>Seçilen şehir: {sehir}</Text>
      <Picker selectedValue={sehir} onValueChange={setSehir}>
        <Picker.Item label="İstanbul" value="istanbul" />
        <Picker.Item label="Ankara" value="ankara" />
      </Picker>
    </View>
  );
}
```

`selectedValue` kontrollü seçili değerdir; `onValueChange` seçim değişince state'i günceller. Yerleşik React Native çekirdeğinde `Picker` artık bulunmaz; doğru paketten import et.

11. `RadioButton` — tek seçenek (Pressable ile)

Çekirdek RN'de hazır `RadioButton` bulunmadığından, tek seçim için state + `Pressable` kullanabiliriz:

```
const [secim, setSecim] = useState('kolay');
{['kolay', 'orta', 'zor'].map(seviye => (
  <Pressable key={seviye} onPress={() => setSecim(seviye)}
    style={{ padding: 12, flexDirection: 'row', alignItems: 'center' }}>
    <Text>{secim === seviye ? '🔘' : '⚪'} {seviye}</Text>
  </Pressable>
))}
```

Tek bir `secim` değeri olduğu için aynı anda yalnızca bir seçenek aktif olabilir. Daha kapsamlı ve erişilebilir radyo kontrolleri için desteklenen UI kütüphanesi de kullanılabilir.

12. `Checkbox` — çoklu seçim veya onay (ek paket)

Snack **Dependencies** içine `expo-checkbox` ekle:

```
import React, { useState } from 'react';
import { View, Text } from 'react-native';
import Checkbox from 'expo-checkbox';

export default function App() {
  const [onay, setOnay] = useState(false);
  return (
    <View style={{ flex: 1, justifyContent: 'center', padding: 30, gap: 15 }}>
      <Checkbox value={onay} onValueChange={setOnay} color={onay ? '#2563EB' : undefined} />
      <Text>{onay ? 'Onaylandı' : 'Henüz onaylanmadı'}</Text>
    </View>
  );
}
```

`value`: true/false; `onValueChange`: kullanıcı değiştirdiğinde çalışır. `RadioButton` **tek seçenek**; checkbox **birden çok bağımsız seçenek** için uygundur.

13. TAM UYGULAMA 1 — Arama + liste + silme (`App.js`)

Bu örnek **yalnızca React ve react-native** kullanır. Expo Snack'e doğrudan yapıştır. `TextInput`, `Button`, `FlatList`, `Pressable`, `useState` birleşir.

```
import React, { useState } from 'react';
import {
  View, Text, TextInput, Button, FlatList, Pressable, StyleSheet,
} from 'react-native';

const ilkKisiler = [
  { id: '1', ad: 'Ünal' },
  { id: '2', ad: 'Ayşe' },
  { id: '3', ad: 'Can' },
];

export default function App() {
  const [kisiler, setKisiler] = useState(ilkKisiler);
  const [arama, setArama] = useState('');
  const [yeniAd, setYeniAd] = useState('');

  const gorunen = kisiler.filter(k =>
    k.ad.toLocaleLowerCase('tr-TR').includes(arama.toLocaleLowerCase('tr-TR'))
  );

  function ekle() {
    if (!yeniAd.trim()) return;
    setKisiler(eski => [...eski, { id: String(Date.now()), ad: yeniAd.trim() }]);
    setYeniAd('');
  }
  function sil(id) {
    setKisiler(eski => eski.filter(k => k.id !== id));
  }

  return (
    <View style={styles.ekran}>
      <Text style={styles.baslik}>Kişi Listesi</Text>
      <TextInput style={styles.girdi} placeholder="Kişi ara..."
        value={arama} onChangeText={setArama} />
      <TextInput style={styles.girdi} placeholder="Yeni kişi adı"
        value={yeniAd} onChangeText={setYeniAd} />
      <Button title="Kişi ekle" onPress={ekle} />
      <FlatList
        data={gorunen}
        keyExtractor={item => item.id}
        ListEmptyComponent={<Text>Kayıt bulunamadı</Text>}
        renderItem={({ item }) => (
          <View style={styles.satir}>
            <Text style={styles.ad}>{item.ad}</Text>
            <Pressable onPress={() => sil(item.id)}>
              <Text style={styles.sil}>SİL</Text>
            </Pressable>
          </View>
        )}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  ekran: { flex: 1, padding: 24, paddingTop: 65, gap: 12 },
  baslik: { fontSize: 27, fontWeight: 'bold', marginBottom: 6 },
```

```
  girdi: { borderWidth: 1, borderColor: '#94A3B8', padding: 12, borderRadius: 10 },
  satir: { padding: 14, flexDirection: 'row', justifyContent: 'space-between',
    alignItems: 'center', backgroundColor: '#EFF6FF', marginTop: 8, borderRadius: 10 },
  ad: { fontSize: 17 },
  sil: { color: '#DC2626', fontWeight: 'bold' },
});
```

Neyi öğrenirsin? Ekran başlığı, kontrollü input, anlık filtreleme, yeni öğe ekleme, `keyExtractor`, öğe silme. `Date.now()` sınıf içi küçük örneklerde kimlik için işe yarar; gerçek kalıcı veride benzersiz ID stratejisi kullanılmalıdır.

14. TAM UYGULAMA 2 — Buton + görsel + Switch benzeri kontrol

Bu örnek ekstra paket olmadan çalışır. Özellikle `TouchableOpacity`, `Image`, `Text` ve koşullu gösterimi birlikte çalıştırır.

```
import React, { useState } from 'react';
import { View, Text, Image, TouchableOpacity, StyleSheet } from 'react-native';

export default function App() {
  const [begeni, setBegeni] = useState(false);
  const [puan, setPuan] = useState(0);
  return (
    <View style={styles.ekran}>
      <Image source={{ uri: 'https://reactnative.dev/img/tiny_logo.png' }}
        style={{ width: 100, height: 100 }} resizeMode="contain" />
      <Text style={styles.baslik}>React Native Kartı</Text>
      <Text>Puan: {puan}</Text>
      <TouchableOpacity style={styles.buton} onPress={() => setPuan(x => x + 1)}>
        <Text style={styles.butonYazi}>+1 PUAN</Text>
      </TouchableOpacity>
      <TouchableOpacity onPress={() => setBegeni(x => !x)}>
        <Text style={{ fontSize: 18 }}>{begeni ? '❤️ Beğenildi' : '🤍 Beğen'}</Text>
      </TouchableOpacity>
    </View>
  );
}
const styles = StyleSheet.create({
  ekran: { flex: 1, alignItems: 'center', justifyContent: 'center', gap: 18 },
  baslik: { fontSize: 24, fontWeight: 'bold' },
  buton: { backgroundColor: '#2563EB', padding: 16, borderRadius: 12 },
  butonYazi: { color: '#fff', fontWeight: 'bold' },
});
```

15. Bileşen seçme kararı

İhtiyaç	En uygun bileşen
Bir başlık göstermek	`Text`
İçeriği iki sütuna bölmek	`View` + Flexbox
Tek seferlik tıklama	`Button`

İhtiyaç	En uygun bileşen
Özel renk/boyutlu buton	`Pressable / TouchableOpacity`
Ad, e-posta, şifre almak	`TextInput`
Kullanıcı resmini göstermek	`Image`
6 kartı kaydırmak	`ScrollView`
500 kişi listelemek	`FlatList`
iPhone çentiğine denk gelmemek	Safe area context `SafeAreaView`
Şehir seçmek	`Picker`
Seviye seçmek (bir tane)	RadioButton yaklaşımı
Sözleşmeyi onaylatmak	`Checkbox`

16. Sık hatalar / sınav tuzakları

H1. `FlatList`'e `data` verip `renderItem` vermemek → öğeler çizilmez.
H2. Uzun listeyi `ScrollView` ile şişirmek → gereksiz render, düşük performans.
H3. `Image` için `width/height` belirtmemek → uzak resim görünmeyebilir.
H4. `TextInput` değerini `onChangeText` ile state'e yazmamak → kontrollü giriş çalışmaz.
H5. `Button` içine `<Text>` çocuk koymak → RN `Button` `title` prop'u kullanır.
H6. `Picker` veya `Checkbox`'ı `react-native`'den import etmek → doğru dış paketi kullan.
H7. RadioButton'ı standart `react-native` export'u sanmak → özel Pressable/UI kütüphanesi gerekir.
H8. `SafeAreaView` sürümüne bakmamak → modern safe-area-context paketi önerilir.

17. Kendini test et (cevap anahtarlı)

S1. Çok sayıda kişiyi performanslı listelemek için? **C:** `FlatList`.

S2. Kullanıcının yazdığı metni hangi prop ile alırsın? **C:** `onChangeText`.

S3. Buton basıldığında çalışan prop? **C:** `onPress`.

S4. Bir resmin tamamı kırpılmadan gözüksün istiyorsan? **C:** `resizeMode="contain"`.

S5. Bir tane seçenek mi, çoklu seçenek mi? **C:** Tek: RadioButton; çoklu/bağımsız: Checkbox.

S6. `keyExtractor` neden? **C:** Liste öğelerini tutarlı biçimde kimliklendirmek.

S7. `View`'de iki kutuyu yan yana nasıl koyarsın? **C:** `flexDirection: 'row'`.

S8. `Button` rengini ve içine ikonlu özel tasarımı rahatça yapmak için? **C:** `Pressable` / `TouchableOpacity` + `Text` / `Image`.

S9. Expo Snack'te dış paket gerekiyorsa nereden eklenir? **C:** `Dependencies` bölümünden.

S10. `SafeAreaProvider` niçin? **C:** Güvenli alan bilgisi için kapsayıcı sağlar.

18. Son dakika kopyala-bak tablosu

Kod	Amacı
`<Text>Yazı</Text>`	Ekrana yazı
`<View style={{flex:1}}>`	Ana kapsayıcı
`<Button title="Bas" onPress={islem} />`	Hazır buton
`<TextInput value={x} onChangeText={setX} />`	Metin girişi
`<Image source={{uri:url}} style={{width:90,height:90}} />`	İnternet resmi
`<ScrollView>...</ScrollView>`	Kaydırma
`<FlatList data={a} renderItem={({item}) => ...} keyExtractor={x => x.id} />`	Liste
`{aktif && <Text>Görünür</Text>}`	Koşullu görünüm
`setListe(l => l.filter(x => x.id !== id))`	Öğeyi silme
`setListe(l => [...l, eleman])`	Öğeyi ekleme

Kaynak dosyalar: W6/README.md, W6/components klasörü.
Bu dosya kaynak ders konularına göre hazırlanmış Türkçe açıklamalı çalışmadır. Kodları Expo Snack'e kopyalarken "tam App.js" örneklerini birbirinden ayrı dene.